Interpolation

- We know that
  - $f(n) \in \theta\big(g(n)\big)$, for $n = b^k$ $(k > 0, b > 1)$
- Show that
  - $f(n) \in \theta\big(g(n)\big)$, for $n \in \mathbb{N}$

$W(n) \in \theta(\log n)$, for $n = 2^k - 1$

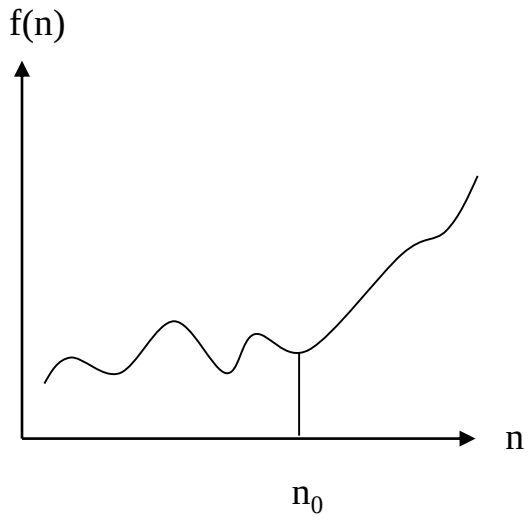


$W(n) \in \theta(\log n)$, for $n \in \mathbb{N}$

Interpolation

- Definitions
  - $f(n)$ is eventually nondecreasing

There exists an integer n_0 such that $f(n) \leq f(m)$, whenever $n_0 \leq n \leq m$



Interpolation

- Definitions
  - $f(n)$ is θ-invariant under scaling
    $f(c * n) \in \theta(f(n))$ for any positive constant c
  - $\theta(f(n))$ is a scale invariant class
  - Example: $f(n) = n^2$
    $f(c * n) = (cn)^2 = c^2$

Polynomial functions & Logarithmic functions → θ-invariant
Exponential functions → not θ-invariant

  - Example: $f(n) = \log n$
    $f(c * n) = \log cn = \log c + \log n \in \theta(\log n)$ ✓
  - Example: $f(n) = 2^n$
    $f(c * n) = 2^{cn} \notin \theta(2^n)$ ✗

$$\lim_{n\to\infty} \frac{2^{cn}}{2^n} = \lim_{n\to\infty} 2^{cn-n} = \lim_{n\to\infty} 2^{n(c-1)} \neq constant$$

Interpolation

- Proposition

  - Suppose that $f(n) \in \theta\big(g(n)\big)$, for $n = b^k$ $(k > 0, b > 1)$. If
    - $f(n)$ is eventually nondecreasing
    - $g(n)$ is eventually nondecreasing
    - $g(n)$ is θ-invariant under scaling

    then

    $f(n) \in \theta\big(g(n)\big)$, for $n \in \mathbb{N}$

  - Example:

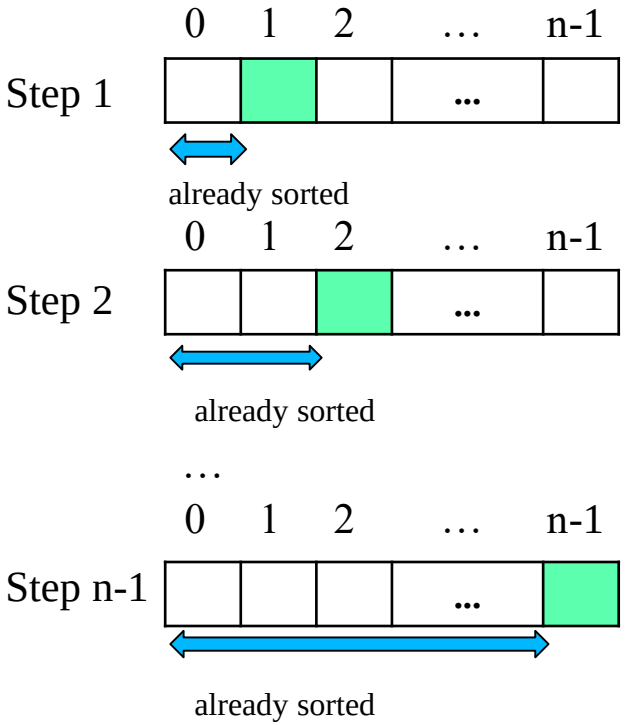
    $W(n) = \log_2(n+1) \in \theta(\log n)$, for $n = 2^k - 1$

    - $\log_2(n+1)$ is eventually nondecreasing
    - $\log n$ is eventually nondecreasing
    - $\log n$ is θ-invariant under scaling

    $\Rightarrow W(n) \in \theta(\log n)$, for $n \in \mathbb{N}$

Insertion Sort

```
function InsertionSort (L[0:n-1])
   for i←1 to n-1 do
      current←L[i]
      position←i-1
      while (position≥0) and (current<L[position]) do
         L[position+1]←L[position]
         position←position-1
      endwhile
      L[position+1]←current
   endfor
end
```



Insertion Sort

```
function InsertionSort (L[0:n-1])
   for i←1 to n-1 do
      current←L[i]
      position←i-1
      while (position≥0) and (current<L[position]) do
         L[position+1]←L[position]
         position←position-1
      endwhile
      L[position+1]←current
   endfor
end
```

- $B(n)$
  - At each iteration i:

    $Current$ (i.e. $L[i]$) $\geq L[position]$ (i.e. $L[i-1]$)

    Only 1 comparison

    $\Rightarrow B(n) = \sum_{i=1}^{n-1} 1 = n - 1 \in \theta(n)$

Insertion Sort

```
function InsertionSort (L[0:n-1])
   for i←1 to n-1 do
      current←L[i]
      position←i-1
      while (position≥0) and (current<L[position]) do
         L[position+1]←L[position]
         position←position-1
      endwhile
      L[position+1]←current
   endfor
end
```

- $W(n)$
  - At each iteration i:

    $Current$ (i.e. $L[i]$) $< L[0: i-1]$)

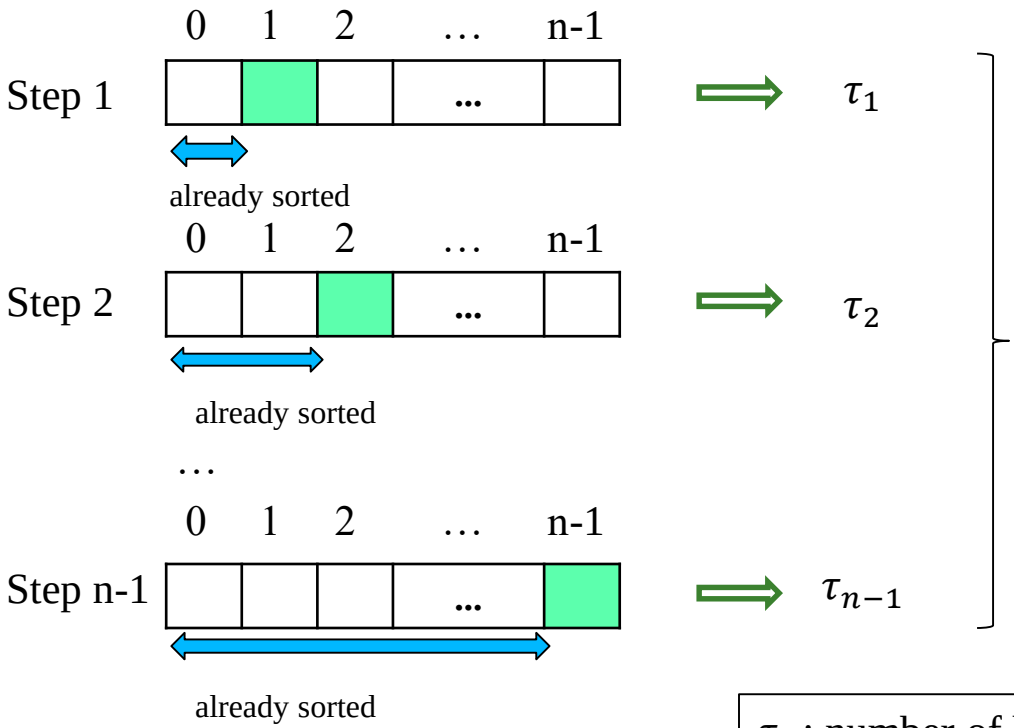
    i comparisons

    $\Rightarrow W(n) = \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} \in \theta(n^2)$

Insertion Sort

- $A(n)$

```
function InsertionSort (L[0:n-1])
  for i←1 to n-1 do
    current←L[i]
    position←i-1
    while (position≥0) and (current<L[position]) do
      L[position+1]←L[position]
      position←position-1
    endwhile
    L[position+1]←current
  endfor
end
```



τ_1

τ_2

τ_{n-1}

τ

τ_i : number of basic operations at step i

$$\tau = \tau_1 + \tau_2 + \cdots + \tau_{n-1}$$

Insertion Sort

```
function InsertionSort (L[0:n-1])
  for i←1 to n-1 do
    current←L[i]
    position←i-1
    while (position≥0) and (current<L[position]) do
      L[position+1]←L[position]
      position←position-1
    endwhile
    L[position+1]←current
  endfor
end
```

- $A(n)$

Average ↔ Expected value

- $A(n) = E[\tau]$

$$= E[\tau_1] + E[\tau_2] + \cdots + E[\tau_{n-1}]$$

$$= \sum_{i=1}^{n-1} E[\tau_i]$$

A very frequent approach when the algorithm can be divided into consecutive steps

Insertion Sort

- $A(n)$
  - $A(n) = E[\tau] = \sum_{i=1}^{n-1} E[\tau_i]$

$$E[\tau_i] = \sum_{j=1}^{i} j * P(\tau_i = j)$$

```
function InsertionSort (L[0:n-1])
  for i←1 to n-1 do
    current←L[i]
    position←i-1
    while (position≥0) and (current<L[position]) do
      L[position+1]←L[position]
      position←position-1
    endwhile
    L[position+1]←current
  endfor
end
```

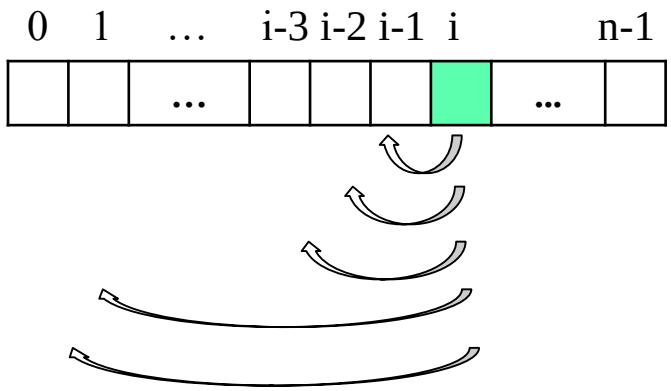
$$A(n) = \sum_{k=B(n)}^{W(n)} k * p_k$$

Insertion Sort

- $A(n)$

  - $A(n) = E[\tau] = \sum_{i=1}^{n-1} E[\tau_i]$

$$E[\tau_i] = \sum_{j=1}^{i} j * P(\tau_i = j)$$




```
function InsertionSort (L[0:n-1])
  for i←1 to n-1 do
    current←L[i]
    position←i-1
    while (position≥0) and (current<L[position]) do
      L[position+1]←L[position]
      position←position-1
    endwhile
    L[position+1]←current
  endfor
end
```

$L[i-1] \leq L[i] \Rightarrow$ 1 comparison
$L[i-2] \leq L[i] < L[i-1] \Rightarrow$ 2 comparisons
$L[i-3] \leq L[i] < L[i-2] \Rightarrow$ 3 comparisons
…
$L[0] \leq L[i] < L[1] \Rightarrow$ i comparisons
$L[i] < L[0] \Rightarrow$ i comparisons

Insertion Sort

```
function InsertionSort (L[0:n-1])
  for i←1 to n-1 do
    current←L[i]
    position←i-1
    while (position≥0) and (current<L[position]) do
      L[position+1]←L[position]
      position←position-1
    endwhile
    L[position+1]←current
  endfor
end
```

- $A(n)$

  - $A(n) = E[\tau] = \sum_{i=1}^{n-1} E[\tau_i]$

$$E[\tau_i] = \sum_{j=1}^{i} j * P(\tau_i = j)$$

$i+1$ intervals
Assume: each equally likely

$L[i-1] \leq L[i] \Rightarrow$ 1 comparison
$L[i-2] \leq L[i] < L[i-1] \Rightarrow$ 2 comparisons
$L[i-3] \leq L[i] < L[i-2] \Rightarrow$ 3 comparisons
…
$L[0] \leq L[i] < L[1] \Rightarrow$ i comparisons
$L[i] < L[0] \Rightarrow$ i comparisons

$$\Rightarrow P(\tau_i = j) = \begin{cases} \frac{1}{i+1}, 1 \leq j \leq i-1 \\ \frac{2}{i+1}, j = i \end{cases}$$

Insertion Sort

```
function InsertionSort (L[0:n-1])
  for i←1 to n-1 do
    current←L[i]
    position←i-1
    while (position≥0) and (current<L[position]) do
      L[position+1]←L[position]
      position←position-1
    endwhile
    L[position+1]←current
  endfor
end
```

- $A(n)$
  - $A(n) = E[\tau] = \sum_{i=1}^{n-1} E[\tau_i]$

$$
\begin{aligned}
E[\tau_i] &= \sum_{j=1}^{i} j * P(\tau_i = j) \\
&= \left[\sum_{j=1}^{i-1} j * P(\tau_i = j)\right] + i * P(\tau_i = i) \\
&= \left[\sum_{j=1}^{i-1} \frac{j}{i+1}\right] + \frac{2i}{i+1} \\
&= \left[\sum_{j=1}^{i} \frac{j}{i+1}\right] + \frac{i}{i+1} \\
&= \frac{i(i+1)}{2(i+1)} + \frac{i}{i+1} \\
&= \frac{i}{2} + 1 - \frac{1}{i+1}
\end{aligned}
$$

Insertion Sort

```
function InsertionSort (L[0:n-1])
  for i←1 to n-1 do
    current←L[i]
    position←i-1
    while (position≥0) and (current<L[position]) do
      L[position+1]←L[position]
      position←position-1
    endwhile
    L[position+1]←current
  endfor
end
```

- $A(n)$

  - $A(n) = E[\tau] = \sum_{i=1}^{n-1} E[\tau_i]$

    $= \sum_{i=1}^{n-1} \left(\frac{i}{2} + 1 - \frac{1}{i+1}\right)$

    $= \frac{n(n-1)}{4} + (n-1) - \left(\frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}\right)$

    $H(n) - 1$ What is this series?

    $= \frac{n(n-1)}{4} + n - H(n)$

    $\in \theta(\log n)$ $\in \theta(???)$

    $\Rightarrow A(n) \in \theta(n^2)$

Insertion Sort

- Inefficient
  - $\theta(n^2)$
- But, nice properties
  - Stable
  - In-place
  - On-line

    &
  - Efficient for nearly sorted data

Insertion Sort

- Stable sorting algorithm
  - For any two elements *A* and *B* having the same value in the list,
    if the position of *A* is less than the position of *B* in the input list,
    then the position of *A* will be less than the position of *B* in the sorted list
  - Example:

in name order

…	Ali Say	…	Cem Say	…

sort wrt surname



…	…	Ali Say	Cem Say	…

Insertion Sort

- In-place sorting algorithm
  - Only a constant amount of memory is used in addition to that needed for the input list
    - Memory for all types of variables
    - $\theta(1)$
  - Example:

    a sorting algorithm that creates a temporary array of size $n \Rightarrow \theta(n)$

    Not in-place

Insertion Sort

- Online sorting algorithm
  - The entire list need not input to the algorithm in advance

    Elements are input over time; as new elements are given, the algorithm maintains the sorted list
  - Example:

Given 10 – 2 – 8 ⟹ | 2 | 8 | 10 |

Given 7 ⟹ | 2 | 7 | 8 | 10 |

Given 5 – 1 ⟹ | 1 | 2 | 5 | 7 | 8 | 10 |

Given 6 – 12 – 3 ⟹ | 1 | 2 | 3 | 5 | 6 | 7 | 8 | 10 | 12 |

…

Insertion Sort

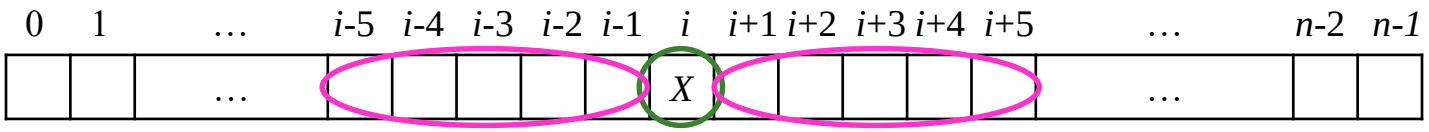
- Efficient for nearly sorted data

  - Given integers n and k ($k \leq n$), suppose $L[0:n-1]$ is a list such that every element in the list is no more than k positions from its stable final position in the sorted list.

    Then, Insertion Sort performs at most $2 * k * (n-1)$ comparisons when sorting $L[0:n-1]$.

    the unique final position of the element
    after any stable sorting algorithm is applied

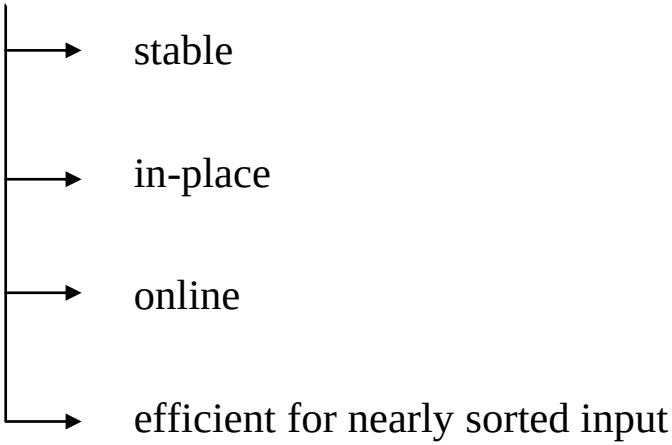
  - Example: $k = 5$



$$2 * 5 * (n-1) \in \theta(n)$$

Insertion Sort

- Insertion sort

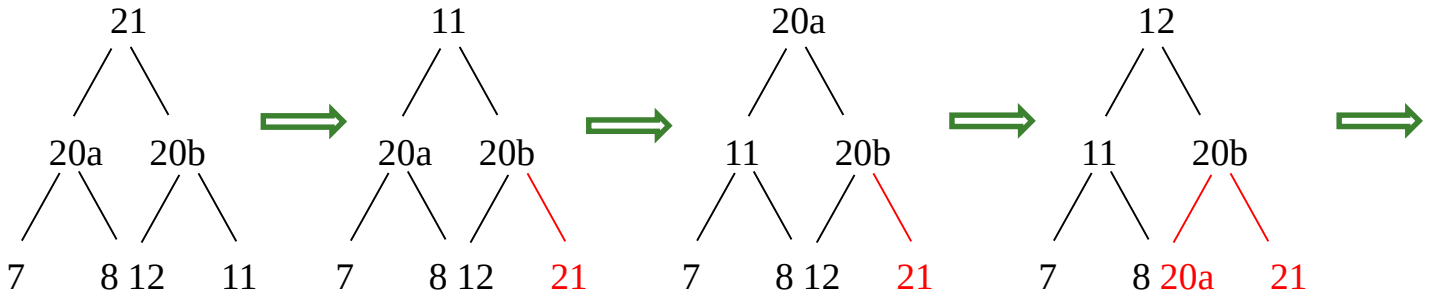


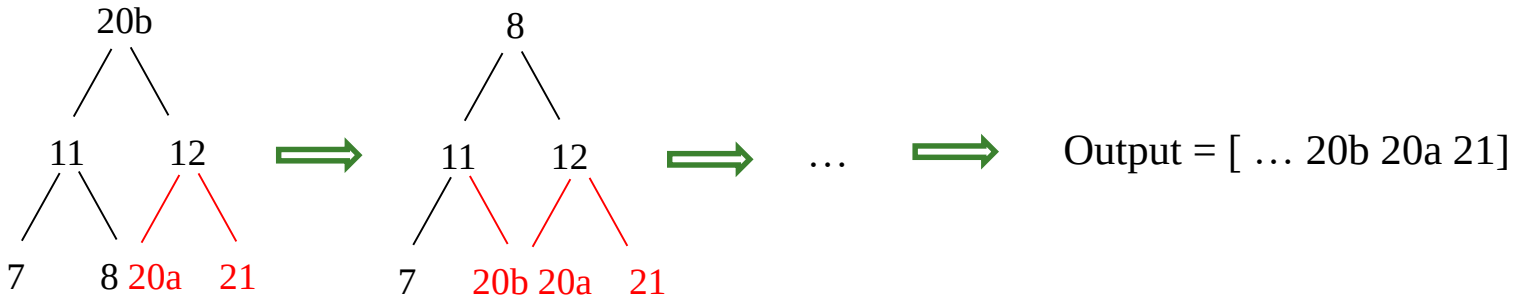
Heapsort

- Not stable
  - Example:

    Input = [12 8 20a 7 21 20b 11]

    [21 20a 20b 12 11 8 7] (after heap is created)





Heapsort

- In-place
  - We use the same array for input and output
  - $\theta(1)$
- Not online
  - Complete heap must be created initially

Adjacent-key Sorting

- Adjacent-key comparison-based sorting
  - Comparisons between list elements are made only between elements that occupy adjacent positions in the list
  - Insertion Sort ???

```
function InsertionSort (L[0:n-1])
   for i←1 to n-1 do
      current←L[i]
      position←i-1
      while (position≥0) and (current<L[position]) do
         L[position+1]←L[position]
         position←position-1
      endwhile
      L[position+1]←current
   endfor
end
```



```
function InsertionSort (L[0:n-1])
   for i←1 to n-1 do
      current←i
      while (current>0) and (L[current]<L[current-1]) do
         interchange (L[current-1],L[current])
         current←current-1
      endwhile
   endfor
end
```



Bingo Sort

- Sorting algorithms
  - General input data
  - No better than $\theta(n \log n)$
- Bingo Sort
  - Repetitions in the input list
  - Better than $\theta(n \log n)$

Bingo Sort

- Input size
  - n : number of elements in the list
  - m : number of distinct elements in the list ($1 \leq m \leq n$)



$f(n, m)$

$f(n) \in \theta(\ldots)$ 

Bingo Sort

- Input:

23	10	15	10	10	23	15	23	23	10

$n = 10$

$m = 3$

  - Initial scan: $\theta(n)$

    min = 10

    max = 23

Bingo Sort

- Input:

23	10	15	10	10	23	15	23	23	10

  - 1st pass – Scan $L[0:9]$
  - Bingo = 10

23	10	15	10	10	23	15	23	23	10
23	10	15	10	10	23	15	23	23	10
10	23	15	10	10	23	15	23	23	10
10	23	15	10	10	23	15	23	23	10
10	10	15	23	10	23	15	23	23	10
10	10	10	23	15	23	15	23	23	10
10	10	10	23	15	23	15	23	23	10
10	10	10	23	15	23	15	23	23	10
10	10	10	23	15	23	15	23	23	10
10	10	10	23	15	23	15	23	23	10

10	10	10	10	15	23	15	23	23	23

shaded:
current scanned element
red:
interchange position

initialize:
current-next-bingo = *max*
if *scanned* ≠ *bingo*
if *scanned* < *current-next-bingo*
current-next-bingo ← *scanned*

⇩

$\theta(n)$

Bingo Sort

- Input:

23	10	15	10	10	23	15	23	23	10

  - 2nd pass – Scan $L[4:9]$
  - Bingo = 15

10	10	10	10	15	23	15	23	23	23
10	10	10	10	15	23	15	23	23	23
10	10	10	10	15	23	15	23	23	23
10	10	10	10	15	15	23	23	23	23
10	10	10	10	15	15	23	23	23	23
10	10	10	10	15	15	23	23	23	23

10	10	10	10	15	15	23	23	23	23

shaded:
current scanned element
red:
interchange position

No pass for *bingo* = *max*
m-1 passes

Bingo Sort

- $B(n,m), W(n,m), A(n,m)$
  - Basic operation

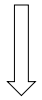
    if *current-scanned-element* = *bingo* then

    Initial comparisons for finding *min* & *max* ✗

    Comparisons to find the next bingo ✗

Bingo Sort

- $B(n,m), W(n,m), A(n,m)$
  - m-1 passes
  - Let n_i : number of times i^{th} smallest element occurs in the list ($1 \le i \le m$)



  - Number of comparisons:

$$\underbrace{(n)}_{1^{\text{st}}\text{ pass}} + \underbrace{(n-n_1)}_{2^{\text{nd}}\text{ pass}} + \underbrace{(n-n_1-n_2)}_{3^{\text{rd}}\text{ pass}} + \cdots + \underbrace{(n-n_1-n_2-\cdots-n_{m-2})}_{(m\text{-}1)^{\text{th}}\text{ pass}}$$

Bingo Sort

- $B(n, m)$
  - Number of comparisons:

    $(n) + (n - n_1) + (n - n_1 - n_2) + \cdots + (n - n_1 - n_2 - \cdots - n_{m-2})$

  - For which values of n_i, this equation is minimized?

    $n_1 = n - m + 1$

    $n_2 = n_3 = \cdots = n_m = 1$

    minimized when *n*=1 & *m*=1 ✗

⇩

$$B(n,m) = (n) + (n - n + m - 1) + (n - n + m - 1 - 1) + \cdots + (n - n + m - 1 - m + 3)$$
$$= (n) + (m - 1) + (m - 2) + \cdots + (2)$$
$$= (n - 1) + \frac{m(m-1)}{2} \in \theta(n + m^2)$$

Bingo Sort

- $W(n,m)$
  - Number of comparisons:

    $(n) + (n - n_1) + (n - n_1 - n_2) + \cdots + (n - n_1 - n_2 - \cdots - n_{m-2})$

  - For which values of n_i, this equation is maximized?

    $n_1 = n_2 = \cdots = n_{m-1} = 1$

    $n_m = n - m + 1$



$$W(n,m) = (n) + (n-1) + (n-2) + \cdots + (n-m+2)$$
$$= (m-1)n - \sum_{i=1}^{m-2} i$$
$$= (m-1)n - \frac{(m-1)(m-2)}{2}$$
$$= \frac{2nm - 2n - m^2 + 3m - 2}{2} \in \theta(n * m)$$

Bingo Sort

- $A(n, m)$
  - Number of comparisons:

    $(n) + (n - n_1) + (n - n_1 - n_2) + \cdots + (n - n_1 - n_2 - \cdots - n_{m-2})$

  - Assume: each of the *m* distinct values occurs *n*/*m* times



$$A(n,m) = (n) + \left(n - \frac{n}{m}\right) + \left(n - 2\frac{n}{m}\right) + \cdots + \left(n - (m-2)\frac{n}{m}\right)$$
$$= (m-1)n - \sum_{i=1}^{m-2} i\frac{n}{m}$$
$$= mn - n - \left(\frac{nm}{2} - \frac{3n}{2} + \frac{n}{m}\right) \in \theta(n * m)$$

$m < \log_2 n \Rightarrow$ better than optimal (general) sorting algorithms

Linear Search with Repeated Elements

- Searching algorithms
  - General input data
  - No better than $\theta(n)$
- Linear Search with repetitions
  - Repetitions in the input list
  - Better than $\theta(n)$
- Input size
  - n : number of elements in the list
  - m : number of distinct elements in the list ($1 \leq m \leq n$)



$f(n, m)$

Linear Search with Repeated Elements

- $A(n, m)$
  - Assume
    - m distinct elements are drawn from $S = \{s_1, s_2, \ldots, s_m\}$
    - Search element $X \in S$
    - It is equally likely that $L[i]$ $(0 \leq i \leq n-1)$ can be any one of the m distinct elements

Linear Search with Repeated Elements

- $A(n,m)$
  - $A(n,m) = \sum_{i=B(n,m)}^{W(n,m)} i * p_i = \sum_{i=1}^{n} i * p_i$

$$p_i = \begin{cases} P(X \notin L[0:i-2]) * P(X = L[i-1]) & , \ if \ \ 1 \le i \le n-1 \\ P(X \notin L[0:n-2]) & , \ if \ \ i = n \end{cases}$$

$$P(X = L[i]) = \frac{1}{m}$$
$$P(X \neq L[i]) = 1 - \frac{1}{m} = \frac{m-1}{m}$$

$$p_i = \begin{cases} \left(\frac{m-1}{m}\right)^{i-1} * \frac{1}{m} & , \ if 1 \le i \le n-1 \\ \left(\frac{m-1}{m}\right)^{n-1} & , \ if \ i = n \end{cases}$$

Linear Search with Repeated Elements

- $A(n,m)$

  - $A(n,m) = \sum_{i=B(n,m)}^{W(n,m)} i * p_i = \sum_{i=1}^{n} i * p_i$

$$= \left[\sum_{i=1}^{n-1} i * \left(\frac{m-1}{m}\right)^{i-1} * \frac{1}{m}\right] + n\left(\frac{m-1}{m}\right)^{n-1}$$

$$\sum_{i=1}^{n-1} i * x^{i-1} = \frac{(n-1)x^n - nx^{n-1} + 1}{(x-1)^2}$$

$$= \frac{1}{m}\left[\frac{(n-1)\left(\frac{m-1}{m}\right)^n - n\left(\frac{m-1}{m}\right)^{n-1} + 1}{(-1/m)^2}\right] + n\left(\frac{m-1}{m}\right)^{n-1}$$

$$= m\left[n\left(\frac{m-1}{m}\right)^n - \left(\frac{m-1}{m}\right)^n - n\left(\frac{m-1}{m}\right)^{n-1} + 1\right] + n\left(\frac{m-1}{m}\right)^{n-1}$$

As $n \to \infty$, goes to 0

As $n \to \infty$, go to 0

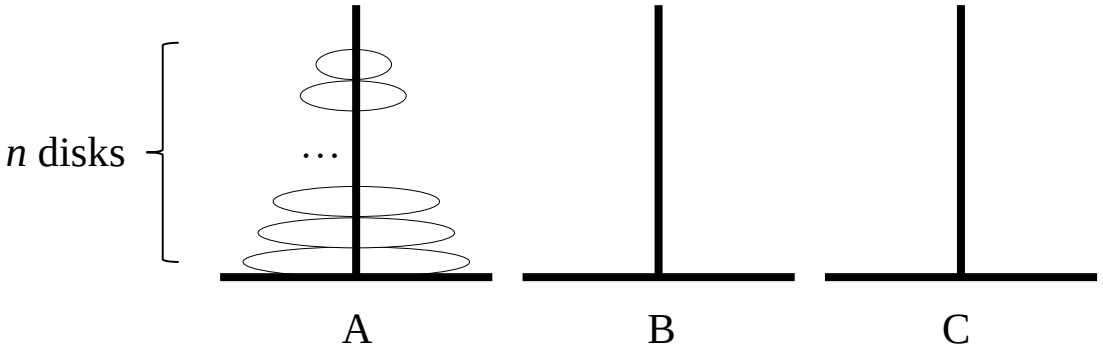
$$\lim_{n\to\infty} nc^n = \lim_{n\to\infty} \frac{n}{c^{-n}} = \lim_{n\to\infty} \frac{1}{-c^{-n}\log c} = \lim_{n\to\infty} \frac{c^n}{-\log c} = 0$$

$$= m$$

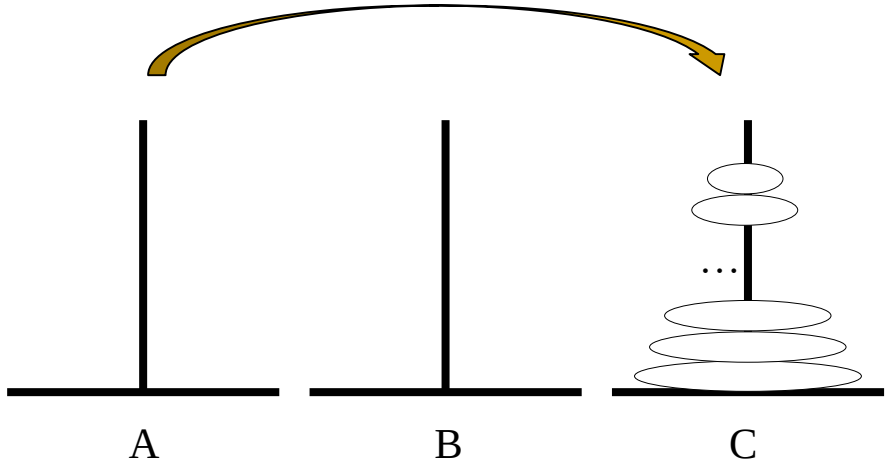
m is fixed $\Rightarrow A(n,m) \in \theta(1)$

Towers of Hanoi

- Initial configuration



- Goal:



One disk at a time
No disk on top of a smaller disk

Towers of Hanoi

- A simulation
  - *n*=5 disks
  - *Video* (*https://youtu.be/34oWQu9RDo0*)

Towers of Hanoi

```
function Towers (A,B,C,n)
   if n=1 then
      write ('Move from' A 'to' C)
   else
      call Towers (A,C,B,n-1)
      write ('Move from' A 'to' C)
      call Towers (B,A,C,n-1)
   endfor
end
```

basic operation?

$A(n), B(n), W(n)$?

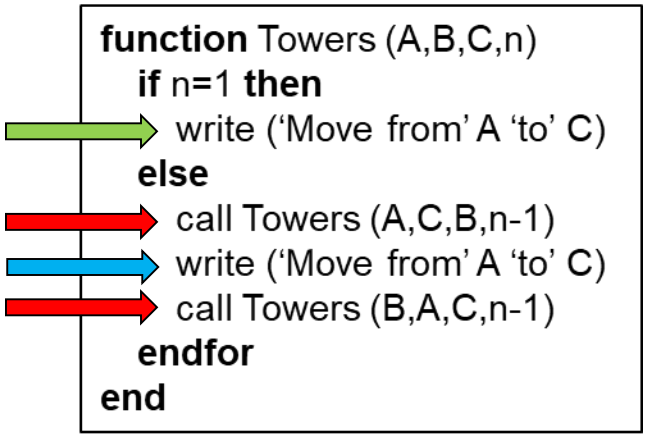
Initial call: Towers (A,B,C,n)

Towers of Hanoi

- Complexity $T(n)$
  - $T(n) = 2 * T(n-1) + 1, \; n > 1$

    $T(1) = 1$

Recurrence relation:
Function on LHS appears also on RHS



Towers of Hanoi

```
function Towers (A,B,C,n)
  if n=1 then
    write ('Move from' A 'to' C)
  else
    call Towers (A,C,B,n-1)
    write ('Move from' A 'to' C)
    call Towers (B,A,C,n-1)
  endfor
end
```

- Complexity $T(n)$

  - $T(n) = 2 * T(n-1) + 1\,,\ n > 1$
    $T(1) = 1$

  - $T(n) = 2 * T(n-1) + 1$
    $= 2 * [2 * T(n-2) + 1] + 1 = 2^2 * T(n-2) + 2 + 1$
    $= 2^2 * [2 * T(n-3) + 1] + 2 + 1 = 2^3 * T(n-3) + 2^2 + 2 + 1$
    $\dots$
    $= 2^{n-1} * T(1) + 2^{n-2} + 2^{n-3} + \dots + 2 + 1$
    $= 2^{n-1} + 2^{n-2} + 2^{n-3} + \dots + 2 + 1$
    $= \sum_{i=0}^{n-1} 2^i$

    $\sum_{i=0}^{n-1} x^i = \frac{x^n - 1}{x-1}\,,\ x \in \mathbb{R}^+, x \neq 1, n > 0$

  $\Rightarrow T(n) = 2^n - 1$

  Exponential order